

Intimauae Store — Client Handoff Guide

Move everything to the client's Supabase + Vercel

This document tells you exactly where to click, what to copy, how to export/import the database, how to set environment variables, and how to re-authenticate admin access.

Project: intimauae (React + Express + Supabase)

Audience: Developer handing off to client production

Date: October 2026

Companion file in repo: `DEPLOY.md`

Table of contents

1. What goes where (architecture)
2. Before you start (checklist at home)
3. Export from YOUR current Supabase
4. Create CLIENT Supabase + import
5. Configure Auth URLs & admin user
6. Push code to GitHub
7. Vercel #1 — Storefront (apps/web)
8. Vercel #2 — API (apps/api)
9. All environment variables (copy tables)
10. Payment gateway (Uniwebpay) reset
11. Update your home / laptop .env
12. Go-live test plan
13. Troubleshooting

1 What goes where

Vercel does **not** host the database. Supabase does. Vercel only hosts the website and API code.

Piece	Where it lives	You click / manage in
Storefront (React)	Vercel project #1	vercel.com → Project (root apps/web)
API (Express)	Vercel project #2	vercel.com → Project (root apps/api)
Postgres database	Client Supabase	supabase.com → Table Editor / SQL
Login / users (Auth)	Client Supabase	supabase.com → Authentication
Secrets / API keys	Vercel Environment Variables	Vercel → Settings → Environment Variables
Payment provider	Uniwebpay dashboard	Notify URL → your live API webhook

Important: Pushing code to GitHub/Vercel does *not* copy products, orders, or settings. You must export/import the database separately.

2 Before you start (on your home PC)

1. Open this project folder: `C:\Users\user\Documents\doll` (or wherever you keep it).

2. Make sure the app still works locally: open a terminal in the project root and run `npm run dev`.
3. Have ready:
 - Client's **Supabase** login (or invite to their org)
 - Client's **Vercel** login (or invite)
 - Client's **GitHub** (or permission to push)
 - Uniwebpay Store ID + private key + Checkout Frames public key
4. Write down (in a password manager, not in chat):
 - Your current Supabase project ref / password
 - Client project will get a **new** DB password — save it immediately when creating the project

Tools to install on your PC (if missing):

Node.js LTS, Git, and optionally `psql` or TablePlus / DBeaver for database dump/restore. Supabase CLI is optional (`npx supabase works`).

3 Export from YOUR current Supabase

Do this from **your** old project (the one you developed on).

3.1 Open your project

1. Go to <https://supabase.com/dashboard>
2. Click your existing project (e.g. intimauae / xmmvprhq...).

3.2 Get your current Database connection string

1. Left sidebar → **Project Settings** (gear icon)
2. Click **Database**
3. Find **Connection string** → tab **URI**
4. Copy the string. Replace `[YOUR-PASSWORD]` with the real DB password.
5. Prefer the **Session** or **Direct** connection for dumping (port `5432` on `db.PROJECT_REF.supabase.co`).

3.3 Export method A — Table dump from dashboard (simplest for small stores)

1. Left sidebar → **Table Editor**
2. For each important table, open it → use export if available, or use SQL Editor (below).
3. Critical tables to migrate:
 - `categories`
 - `products`

- `product_images`
- `warehouses`
- `settings` (CMS, SEO, payment flags)
- `coupons`
- `blog_posts`
- `cms_pages` (if used)
- `orders` / `order_items` / `payments` (only if client wants live order history)
- `profiles` (optional — Auth users are separate)

3.4 Export method B — SQL dump with `pg_dump` (recommended, complete)

On your PC, open PowerShell or Terminal:

```
pg_dump "postgresql://postgres:YOUR_PASSWORD@db.YOUR_OLD_REF.supabase.co:5432/postgres" ^
--no-owner --no-acl -F c -f intimaue-backup.dump
```

Or plain SQL:

```
pg_dump "postgresql://postgres:YOUR_PASSWORD@db.YOUR_OLD_REF.supabase.co:5432/postgres" ^
--no-owner --no-acl -f intimaue-backup.sql
```

Save `intimaue-backup.dump` or `.sql` in a safe folder (Documents/backups). This is your migration file.

3.5 Export Auth users (optional)

Auth users are **not** inside normal SQL tables the same way. Options:

- **Easiest for handoff:** Create a fresh admin user on the client project (recommended).
- **Advanced:** Supabase Dashboard → Authentication → Users → invite/recreate, or use Supabase migration tools for auth.

Password hashes do not always transfer cleanly. Plan to recreate the admin account on the client project unless you use an official auth export.

3.6 Export Storage files (product images)

1. Dashboard → **Storage**
2. Note every **bucket** name.
3. Download important files, or plan to re-upload from your local `uploads` / media folder.
4. If image URLs contain your old project host, you will update them after import (or re-upload so new public URLs are used).

4 Create CLIENT Supabase + import

4.1 Create the new project

1. Log out of your personal Supabase (or switch org).
2. Log into the **client's** Supabase account.
3. Click **New project**
4. Fill in:
 - Organization: client's org
 - Name: e.g. `intimauae-prod`
 - Database password: generate a strong one → **save in password manager**
 - Region: closest to UAE / customers
5. Click **Create new project** → wait until status is Healthy.

4.2 Copy the new API keys

1. **Project Settings** → **API**
2. Copy into a notepad (temporary):
 - **Project URL** → later: `SUPABASE_URL` and `VITE_SUPABASE_URL`
 - **anon public** → `SUPABASE_ANON_KEY` / `VITE_SUPABASE_ANON_KEY`
 - **service_role** → `SUPABASE_SERVICE_ROLE_KEY` (server only — never put in Vite/web)
3. **Project Settings** → **Database** → Connection string URI:
 - Transaction pooler (:6543) → `DATABASE_URL`
 - Direct (db.....:5432) → `DATABASE_URL_DIRECT` (optional fallback)

4.3 Apply the schema (empty structure)

Option A — SQL Editor (no CLI):

1. Client project → left sidebar → **SQL Editor**
2. Click **New query**
3. Open on your PC: [supabase/migrations/20260326120000_init.sql](https://supabase.com/migrations/20260326120000_init.sql)
4. Copy all → paste into SQL Editor → click **Run**
5. Confirm success (no red errors). Check Table Editor for `products`, `settings`, etc.

Option B — CLI from your home PC:

```
cd C:\Users\user\Documents\doll
npx supabase login
npx supabase link --project-ref CLIENT_PROJECT_REF
npm run db:migrate
```

4.4 Import your data

If you used `pg_dump` (.sql):

```
psql "postgresql://postgres:CLIENT_PASSWORD@db.CLIENT_REF.supabase.co:5432/postgres" -f intima
```

If you used custom dump (.dump):

```
pg_restore --no-owner --no-acl -d "postgresql://postgres:CLIENT_PASSWORD@db.CLIENT_REF.supabase
```

If schema already exists, prefer **data-only** restore, or restore into a fresh project. Resolve “already exists” errors carefully — do not drop client Auth tables.

If dump fails — seed CMS policies at least:

1. On your PC, edit `apps/api/.env` to use the **client** `DATABASE_URL`.
2. Run: `node apps/api/scripts/seed-policies.js`
3. Re-create products in Admin UI after deploy, or run `npm run seed` if configured for client DB.

4.5 Recreate Storage buckets

1. Client Supabase → **Storage** → **New bucket**
2. Match old bucket names; set public if product images were public.
3. Upload files.
4. Update image URLs in `product_images` if the hostname changed.

5 Auth URLs & first admin user

5.1 URL configuration

1. Client Supabase → **Authentication** → **URL Configuration**
2. **Site URL:** set to your live storefront, e.g. `https://STORE.vercel.app` (update again after Vercel domain is known)
3. **Redirect URLs** → Add:
 - `https://STORE.vercel.app/**`
 - `https://www.client-domain.com/**` (if custom domain)
 - `http://127.0.0.1:5173/**` (local testing)
4. Save.

5.2 Create admin user

1. **Authentication** → **Users** → **Add user** → Create new user
2. Enter email + password (client's admin email).
3. Go to **Table Editor** → table `profiles`
4. Find the row with that email / user id
5. Set column `role` to **admin** (exact value: admin)
6. Save / confirm the cell updated

After the site is live, log in at `/login` then open `/admin`. Reload must keep you on the same admin page.

6 Push code to GitHub

1. Create a repo under the **client's** GitHub (or transfer yours).
2. On your PC (project root):

```
git status
git add .
git commit -m "Prepare production handoff"
git remote add origin https://github.com/CLIENT_ORG/REPO.git
git push -u origin main
```

Confirm `.env` files are NOT listed in `git status`. They must stay ignored. Never commit secrets.

7 Vercel #1 — Storefront (where customers shop)

1. Open **`https://vercel.com`** → log in as **client**.
2. Click **Add New...** → **Project**
3. Import the GitHub repo.
4. Before Deploy, open **Root Directory** → **Edit** → choose `apps/web` → Continue.
5. Framework Preset: **Vite**
6. Build Command: `npm run build`
7. Output Directory: `dist`
8. Open **Environment Variables** and add every row in the "Web env" table in section 9 (Production + Preview).

9. Click **Deploy**
10. When done, copy the URL (e.g. `https://intimauae-xxxx.vercel.app`) — this is **APP_URL** / storefront.
11. Optional: Project → **Settings** → **Domains** → add client domain → follow DNS instructions.

You can deploy web first with a temporary `VITE_API_URL`, then update it after the API project exists and **Redeploy**.

8 Vercel #2 — API (backend)

1. Vercel → **Add New...** → **Project** again (same repo).
2. Root Directory → `apps/api`
3. This folder already has `vercel.json` + `api/index.js`.
4. Add all variables from the “API env” table in section 9.
5. Especially set:
 - `APP_URL` = storefront URL from step 7
 - `API_URL` = this API project’s URL (after first deploy, update + redeploy if needed)
 - `ADMIN_DEV_BYPASS` = `0`
 - `NODE_ENV` = `production`
6. Click **Deploy**
7. Test: open `https://YOUR-API.vercel.app/api/health` — must show `"ok": true`
8. Return to **web** project → Settings → Environment Variables → set `VITE_API_URL` to the API URL → **Redeploy**

9 Environment variable tables

9.1 Web project (`apps/web`)

Variable name	Where to get value	Example shape
<code>VITE_SUPABASE_URL</code>	Supabase → Settings → API → Project URL	<code>https://xxxx.supabase.co</code>
<code>VITE_SUPABASE_ANON_KEY</code>	Supabase → anon public key	<code>eyJhbGciOi...</code>
<code>VITE_API_URL</code>	Vercel API project URL	<code>https://xxx-api.vercel.app</code>
<code>VITE_CHECKOUT_FRAMES_PK</code>	Checkout.com / Uniwebpay Frames public key	<code>pk_...</code>
<code>VITE_DEFAULT_LANG</code>	Fixed	<code>ar</code>

Variable name	Where to get value	Example shape
VITE_DEFAULT_CURRENCY	Fixed	SGD

9.2 API project (apps/api)

Variable name	Where to get / what to set
NODE_ENV	production
APP_URL	Storefront Vercel URL (and later custom domain)
API_URL	This API's Vercel URL
CORS_ORIGIN	Same as APP_URL; comma-separate extra domains
SUPABASE_URL	Client Project URL
SUPABASE_ANON_KEY	Client anon key
SUPABASE_SERVICE_ROLE_KEY	Client service_role (secret)
DATABASE_URL	Pooler URI :6543 with real password
DATABASE_URL_DIRECT	Optional direct db....:5432
ADMIN_DEV_BYPASS	0 (never 1 in production)
UNIWEBPAY_STORE_ID	Uniwebpay dashboard
UNIWEBPAY_PRIVATE_KEY_PKCS8	PKCS8 private key (secret)
UNIWEBPAY_KEY_VERSION	Usually 1
UNIWEBPAY_BASE_URL	https://app.uniwebpay.com
UNIWEBPAY_NOTIFY_URL	https://API.vercel.app/api/payments/webhook
UNIWEBPAY_WEBHOOK_SECRET	Optional random string
CHECKOUT_FRAMES_PK	Frames public key

How to add on Vercel: Project → **Settings** → **Environment Variables** → for each row: Name + Value → Environments: Production (and Preview if you want) → **Save** → then **Deployments** → : on latest → **Redeploy** .

10 Payment gateway reset (Uniwebpay)

1. Log into Uniwebpay / merchant portal.

2. Set notification / webhook / notify URL to:
`https://YOUR-API.vercel.app/api/payments/webhook`
3. Confirm Store ID and key version match Vercel env.
4. On live site → login as admin → **Admin → Settings → Payments**
5. Paste keys only when changing them. Empty fields keep existing secrets.
6. Save → try a small test checkout.

Security already in code: rate limits, webhook rejects unknown transactions, secrets not shown again after save, admin bypass off in production.

11 Update YOUR home PC after handoff

1. Open `apps/api/.env` — replace all values with **client** Supabase + live URLs (or keep a separate `.env.client` copy).
2. Open `apps/web/.env` — same for `VITE_*` client values.
3. Delete or archive old personal project keys so you don't deploy to the wrong place by mistake.
4. From project root: `npm install` then `npm run dev`
5. Local web: `http://127.0.0.1:5173` — API: `http://127.0.0.1:4123`

```
# apps/web/.env (example shape only)
VITE_SUPABASE_URL=https://CLIENT_REF.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_API_URL=http://127.0.0.1:4123
VITE_CHECKOUT_FRAMES_PK=pk...

# apps/api/.env (example shape only)
APP_URL=http://127.0.0.1:5173
API_URL=http://127.0.0.1:4123
SUPABASE_URL=https://CLIENT_REF.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
DATABASE_URL=postgresql://...
ADMIN_DEV_BYPASS=0
```

12 Go-live test plan

- ☐ Open storefront home page — products load
- ☐ Open /about, /faqs, policies — CMS text visible
- ☐ Register / login works

- ☐ Admin login → /admin stays on same URL after browser reload
- ☐ Account /orders stays after reload
- ☐ Add product + new category from dropdown → appears in DB
- ☐ Add to cart → checkout → payment starts (or sandbox)
- ☐ /api/health returns ok
- ☐ Contact form creates a message in Admin → Messages

13 Troubleshooting

Symptom	Where to click / what to fix
Blank products / settings errors	API DATABASE_URL wrong; check /api/health; try DATABASE_URL_DIRECT
getaddrinfo ENOTFOUND pooler	DNS/network; set DATABASE_URL_DIRECT to db.REF.supabase.co:5432
CORS / blocked API from browser	API env APP_URL and CORS_ORIGIN must match exact storefront origin (https, no trailing slash mismatch)
Login works but /admin → /account	profiles.role not admin; wait for profile load; hard refresh
Payment fails	Admin Audit log; missing UNIWEBPAY_* on Vercel; notify URL wrong
Images 404	Storage bucket / public URLs still point to old project
Vercel build fail on web	Root Directory must be apps/web; env vars missing VITE_*

Final order of operations (print this page)

1. Export old DB (+ storage notes)
2. Create client Supabase → run migration SQL → import data
3. Create admin user → role admin → Auth URLs
4. Push GitHub
5. Vercel web (apps/web) + env
6. Vercel api (apps/api) + env
7. Point VITE_API_URL → Redeploy web
8. Update Uniwebpay notify URL
9. Update Auth Site URL to final domain
10. Test checklist above

11. Update home PC .env to client keys

End of guide — Intimaue client handoff. Keep this PDF with the client credentials vault; do not commit secrets into git.